Programming 7312

Smart-X IoT Part 3

Implementation and Completion Report

ST10438312

Andre Fourie

Group 1

Paper Due Date

November 13, 2026

Contents

1. Introduction 3
2. Implementation Report 4
Architecture of the Network Topology feature 4
Tree – An AVL tree for historical telemetry 5
Graph – Modeling the ESP32 Mesh 7
Graph Traversal: BFS and DFS 8
Minimum spanning tree – Prim's algorithm with a heap 9
Main menu integration 11
Performance and scaling 11
3. Project Completion Report: Challenges and Learnings 12
Challenges and how they were overcome 12
Key Learnings 15
4. Technology Recommendations and Future Scaling 16
Cloud Ingestion Pipeline 17
Security 17
Durable History 17
Container Orchestration 17
Telemetry Analysis and Push 18

1. Introduction

For part 3 of the Smart-X IoT project a Network Topology page was added. It shows what a sensor reads in a given time window, which node can still reach the gateway, and which routes use the least power.

Question the dashboard answers	Structure	Where it lives	Section
What did the sensor read between 08:24 and 08:26?	AVL tree with a pruned range query	SmartX.Shared/ Collections/ AvlTree.cs, used by TelemetrTimelineStore	2.2
Which nodes are linked to the gateway, and which are down?	Weighted undirected graph, BFS, and DFS	SmartX.Api/ Topology / MeshGraph.cs, MeshAlgorithms.cs	2.3, 2.4
What is the cheapest way for each node to reach the gateway	Prim's minimum spanning tree on a binary heap	MeshAlgorithms.cs	2.5
How do the three appear to the operator?	Network Topology page in the unified menu	SmartX.Client/View/ TopologyView.xaml	2.6

The project is a .NET 10 app with three projects inside of it being SmartX.Api, SmartX.Client, and SmartX.Shared.

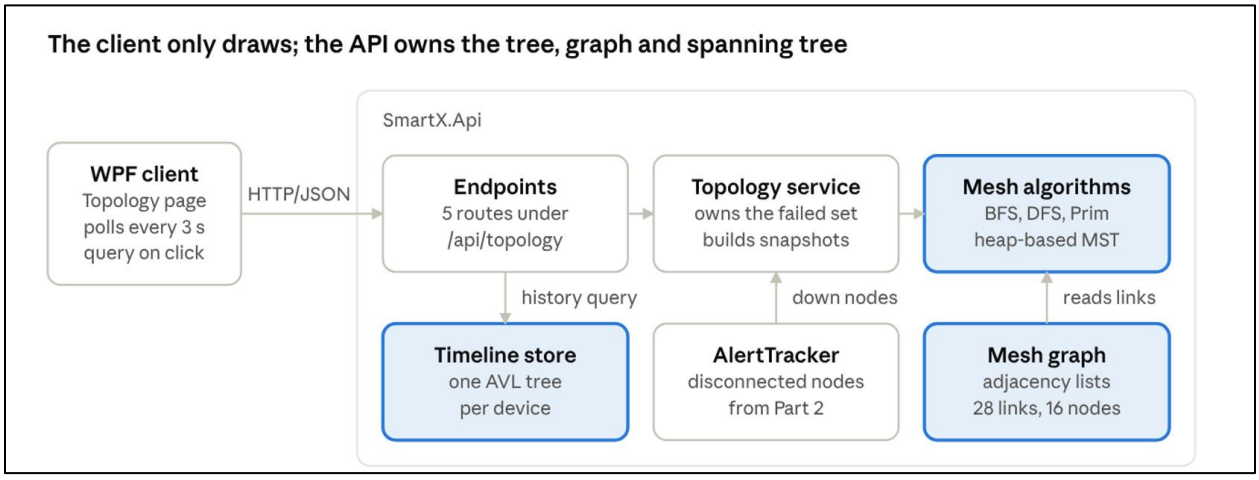
Project Links:

GitHub Link -> https://github.com/AndreFourie12/SmartX_IoT.git

2. Implementation Report

Architecture of the Network Topology feature

The API is responsible for holding every data structure and the WPF client only draws what the API returns, meaning the tree and graph are built once, every client sees the same data, and both can be tested without a user interface. Telemetry goes into TelemetryTimelineStore (one AVL tree per device), and network health goes through MeshTopologyService, which contains the graph and the set of nodes the operator switched off. A node is counted as being down if it is in that set or if Part 2's AlertTracker reports it as disconnected. The page talks to the API through five routes under /api/topology.



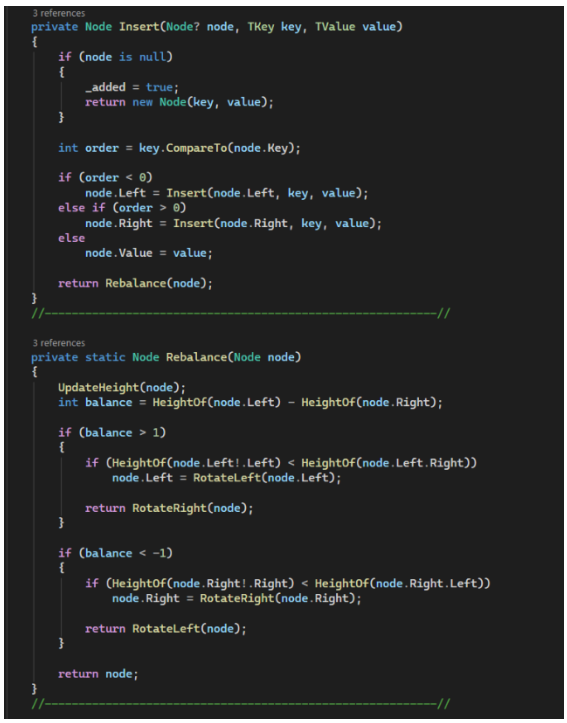
Tree – An AVL tree for historical telemetry

The tree is responsible for storing each sensors readings in timestamp order, so a query like “all moisture readings for ESP32-SOIL-02 between 08:24:00 and 08:26:40” goes directly to that window instead of reading the entire history.

Using a plain binary search tree would not be enough here. Readings arrive in order of time, so each new reading becomes the right child of the previous ones, and the tree turns into a linked list of height n. Because of this a lookup then costs 0(n) (Cormen, et al., 2022). A self-balancing tree keeps the height at 0(log n) for whatever order the data arrives in.

I chose an AVL tree over a red-black tree because its height is at most 1.44log2(n+2) (Adelson-Velsky & Landis, 1962), against 2log2(n+1) for red-black (Sedgewick & Wayne, 2011). Red-black trees rotate less on insert, but each sensor adds only one reading every few second while operators can run many range queries, so the shallower tree is the better option in this case. Now it replaces the Part 2 SortedDictionary, where the Range method skipped from the smallest key and cost 0(n), behind the same Append, Range, CountFor, and BuildTimeline methods. The keys are TimelineKey values (timestamp plus the ingest sequence number), so two readings with the same timestamp never overwrite each other. Every device keeps the 5000 newest readings, and RemoveMin drops the oldest in 0(log n).

After every insert the nodes on the path back to the root recalculate their height. If a nodes balance factor reaches 2 or -2, the tree rotates once or twice depending on the shape.



```
3 references
private Node Insert(Node? node, TKey key, TValue value)
{
    if (node is null)
    {
        _added = true;
        return new Node(key, value);
    }

    int order = key.CompareTo(node.Key);

    if (order < 0)
        node.Left = Insert(node.Left, key, value);
    else if (order > 0)
        node.Right = Insert(node.Right, key, value);
    else
        node.Value = value;

    return Rebalance(node);
}
//-------------------------------------------------------//

3 references
private static Node Rebalance(Node node)
{
    UpdateHeight(node);
    int balance = HeightOf(node.Left) - HeightOf(node.Right);

    if (balance > 1)
    {
        if (HeightOf(node.Left!.Left) < HeightOf(node.Left.Right))
            node.Left = RotateLeft(node.Left);

        return RotateRight(node);
    }

    if (balance < -1)
    {
        if (HeightOf(node.Right!.Right) < HeightOf(node.Right.Left))
            node.Right = RotateRight(node.Right);

        return RotateLeft(node);
    }

    return node;
}
//-------------------------------------------------------//
```

The range query in listing 2 is what allows for the lookups to be fast. If a nodes key is below the start of the window, its left subtree is skipped, and if the key is above the end, its right subtree is skipped. The search walks the two edges of the window, returns results that have already been sorted, and reports how many nodes it visited, which is shown on the page under the history table. That gives 0(log n + k) for k results



```
4 references
public int Range(TKey from, TKey to, List<KeyValuePair<TKey, TValue>> into)
{
    int visited = 0;
    Collect(_root, from, to, into, ref visited);
    return visited;
}
//-------------------------------------------------------------//


3 references
private static void Collect(Node? node, TKey from, TKey to, List<KeyValuePair<TKey, TValue>> into, ref int visited)
{
    if (node is null)
        return;

    visited++;

    if (node.Key.CompareTo(from) > 0)
        Collect(node.Left, from, to, into, ref visited);

    if (node.Key.CompareTo(from) >= 0 && node.Key.CompareTo(to) <= 0)
        into.Add(new KeyValuePair<TKey, TValue>(node.Key, node.Value));

    if (node.Key.CompareTo(to) < 0)
        Collect(node.Right, from, to, into, ref visited);
}
//-------------------------------------------------------------//
```

Graph – Modeling the ESP32 Mesh

A mesh is a set of devices joined by wireless links, so it was modelled as a graph. Each ESP32 sensors is a vertex, each radio link is an edge, and the gateway GW-01 is the vertex where every route ends at. The demo mesh has 16 vertices and 28 links. The links are undirected because radio carries traffic both ways, and the cost is weighed in milliwatts per packet. The costs are fixed sample values; a real system would get them from signal strength. AddLink refuses an unknown node, a negative cost or a duplicate link, so a bad link can never reach Prim.

Adjacency lists where used over a matric. A mesh node hears only a few neighbors, so the graph is sparce as a matric needs V x V cells whatever the number of links, while lists need V + E. Each link is stored at both ends.

```
13 references
private readonly Dictionary<string, List<MeshLink>> _adjacency = new(DeviceIdentifier.Comparer);

1 reference
public void AddLink(string a, string b, double costMw)
{
    if (!_adjacency.ContainsKey(a) || !_adjacency.ContainsKey(b))
        throw new ArgumentException($"Both ends of a link must be existing nodes ('{a}' to '{b}').");

    if (double.IsNaN(costMw) || costMw < 0)
        throw new ArgumentOutOfRangeException(nameof(costMw), costMw, "A link's cost cannot be negative.");

    if (IsLinked(a, b))
        throw new InvalidOperationException($"'{a}' and '{b}' are already linked.");

    _links.Add(new MeshLink(a, b, costMw));
    _adjacency[a].Add(new MeshLink(a, b, costMw));
    _adjacency[b].Add(new MeshLink(b, a, costMw));
}

//------------------------------------------------------//

2 references
public IReadOnlyList<MeshLink> LinksOf(string id) => _adjacency[id];
```

The graph never changes when a sensor fails. Every algorithm instead receives an isDown function and skips any node it returns true for, whether the operator switched it off or AlertTracker reports it disconnected.

Graph Traversal: BFS and DFS

Breadth-first search (BFS) starts at the gateway and visits neighbors' level by level using a queue. The first time it reaches a node is by the route with the fewest hops, so it gives each nodes hop count, and any live node it does not reach is cut off from the gateway. Depth-first search (DFS) uses a stack instead; it reaches the same nodes and is offered as a toggle on the page. Both run in 0(V + E) time (Cormen, et al., 2022).

```
0 references
public static BfsResult Bfs(MeshGraph graph, string root, Func<string, bool> isDown)
{
    var order = new List<string>();
    var hops = new Dictionary<string, int>(DeviceIdentifier.Comparer);
    var parent = new Dictionary<string, string>(DeviceIdentifier.Comparer);

    if (isDown(root))
        return new BfsResult(order, hops, parent);

    var queue = new Queue<string>();
    hops[root] = 0;
    queue.Enqueue(root);

    while (queue.Count > 0)
    {
        string current = queue.Dequeue();
        order.Add(current);

        foreach (var link in graph.LinksOf(current))
        {
            if (isDown(link.To) || hops.ContainsKey(link.To))
                continue;

            hops[link.To] = hops[current] + 1;
            parent[link.To] = current;
            queue.Enqueue(link.To);
        }
    }

    return new BfsResult(order, hops, parent);
}
//------------------------------------------------------------//
```

Minimum spanning tree – Prim's algorithm with a heap

BFS finds the path with the fewest hops, but a long weak link costs far more energy than a short strong one, and ESP32 sensors run on batteries. The minimum spanning tree (MST) connects every node to the gateway as the lowest total link cost, so routing along it is the most power-efficient way to keep the mesh connected.

Prim's algorithm grows the tree outward from the gateway, where it repeatedly adds the cheapest link that joins a tree node to a non-tree node (Prim, 1957). A min-priority queue keeps the candidate links ordered by costs. .NETS PriorityQueue<TElement, TPriority> is an array-backend quaternary min-heap (.NET Foundation, n.d.) with 0(log n) enqueue and dequeue and it is the same collection that is used for Part 2 alert queue (Cormen, et al., 2022).

```
0 references
public static PrimResult Prim(MeshGraph graph, string root, Func<string, bool> isDown)
{
    var edges = new List<MeshLink>();
    var parent = new Dictionary<string, string>(DeviceIdentifier.Comparer);
    var inTree = new HashSet<string>(DeviceIdentifier.Comparer);
    double total = 0;

    if (!isDown(root))
    {
        var queue = new PriorityQueue<(string From, string To), double>();
        inTree.Add(root);
        QueueLinks(graph, root, inTree, isDown, queue);

        while (queue.TryDequeue(out var step, out double cost))
        {
            // A cheaper link may already have brought this node in.
            if (!inTree.Add(step.To))
                continue;

            edges.Add(new MeshLink(step.From, step.To, cost));
            parent[step.To] = step.From;
            total += cost;

            QueueLinks(graph, step.To, inTree, isDown, queue);
        }
    }

    var unreachable = new List<string>();
    foreach (var node in graph.Nodes)
    {
        if (!isDown(node.Id) && !inTree.Contains(node.Id))
            unreachable.Add(node.Id);
    }

    return new PrimResult(edges, total, parent, unreachable);
}
```

Here the cheapest link crosses the boundary between the tree and the rest of the graph always belongs to some minimum spanning tree. Prim was chosen over Kruskal (Kruskal, 1956) because it grows from the gateway, so its parent map gives each node's next hop to the server. Each link enters the heap at the most twice, so it runs in 0(E log E). The tree minimizes the power of the whole network, not of each node's own path, so Dijkstra

algorithm was also added, which uses the same priority queue and finds each node's cheapest route (Dijkstra, 1959).

RouteTo follows the parent map back to the gateway. For ESP32-SOIL-02 with every node, the tree route is SOIL-02, PMP-01, SOILD-01, GW-01, these three hops cost 9.2 mW. BFS would use two hops costing 10.9 mW, because the direct PMP-01 to gateway link costs 7.4 mW. With Pump A down, the tree route is 5 hops at 15.7 mW, BFS gives 4 hops at 16.1 mW, and Dijkstra finds 4 hops at 13.8 mW. The page's Route card lists all three, with the cheapest in green.

The table shows the result on the demo mesh:

State	Live Links	Cost of every live link	MST links	MST cost	Saving
All nodes up	28	99.2 mW	15	41.3 mW	57.9 mW (58%)
Pump A down	24	83.8 mW	14	40.0 mW	43.8 mW (52%)

Main menu integration

The network topology page which was locked in part 1 and a blank page with placeholder text in part 2, is now a fully functional page that opens with Ctrl+3. The view is created once and reused, so the selected node and last history query survives when switching between pages. API calls are waited for, so the interface thread never blocks anything, and the three-second poll runs only while the tab is visible. Nothing was changed in parts 1 and 2 code wise when the tree replaced the old store, because its public methods did not change.

Performance and scaling

In table 1 it shows the range query on the running gateway (a 7 minute window on ESP32-SOIL-02) and in a stress test with 60,000 points. In both cases the tree is well inside the AVL height bound, and the query visits only 10 to 20 more nodes than the readings it returns. The old method would have skipped every earlier reading first.

Readings stored (n)	**Height bound 1.44log2(n+2)**	**Measured tree height**	**Readings returned (k)**	**Nodes visited**
4,047 (running gateway)	17.3	12	105	117
60,000 (stress test)	22.9	19	100	116

Removing 20,000 of the 60,000 points left the height at 18, so deletes keep the tree balanced as well. The visited count in the history card comes straight from Range, so the first row can be reproduced in the demo.

BFS costs 0(V + E), and Prim and Dijkstra cost 0(E log E) with the heap behind PriorityQueue. A test builds a random connected 2000-node mesh where Prim returns 1999 edges (V – 1), BFS reaches all 2000 nodes, and no nodes Dijkstra cost is above its tree route cost. The mesh gets recomputed on every request which works fine at 16 nodes. For a larger mesh the tree could be cached and recomputed only when a link or node changes state.

3. Project Completion Report: Challenges and Learnings

Part	What was built	Main Structure	What it taught
Part 1: Ingestion gateway	Sensor registration with MAC address, location and category, AES-256 encrypted attachments, and the Telemetry Pulse health gauge	TelemetryPacket<T> without boxing, operator overloading, jagged arrays into List<T> and a recursive deployment validator.	Mixed value types must be handled without boxing, so the generic packet is constrained to struct.
Part 2: Real-time processing	A two-lane ingestion pipeline, a command stream with undo, search and filter, and suggested actions.	Queue, PriorityQueue, Stack, Dictionary, SortedDictionary, and HashSet	These collections are not thread-safe, so each class that owns one guards it with a short lock and never holds it while calling another class
Part 3: Network Topology	Historical range queries, mesh routing and node-failure demonstrations.	AVL tree, graph, BFS and DFS, Prim and Dijkstra	A new structure can sit behind an existing interface without breaking the other parts.

Challenges and how they were overcome

Slow stores that look fast.

Part 2 store used a sorted collection, so the ranged queries look solved. Readings its Ranged method showed that it was skipping entries from the smallest key, so cost grew with the age of the data. This was addressed by writing an AVL tree where the range query prunes the whole subtree, and it was made to return the nodes visited so the gain shows on screen.

Replacing a structure without breaking other parts.

The store feeds the Part 1 charts and the Part 2 alerts, so Append, Range, CountFor, and BuildTimeline were left unchanged and swapped only what sits behind them. The existing TimelineKey already carried a sequence number, which gave the tree unique keys for identical timestamps.

Getting “Down” from two sources

A node can be down either due to the operator simulating a failure or because AlertTracker saw it disconnect. Passing every algorithm one isDown function avoided duplicating that login, this way real and simulated failures behaved the same. The two sources also have to agree across pages, so simulated failure now also silences the node in the simulated fleet, and restoring it start the node reporting again, which makes Command stream show the same node go offline.

Shortest route not the cheapest

The route card showed that a node’s route through the spanning tree isn’t always the cheapest one. With Pump A down, Moisture Sensor B takes 5 hops at 15.7 mW by the tree, 4 hops at 16.1 mW by BFS and 4 hops at 13.8 by Dijkstra. The tree minimizes the power of the whole network, so Dijkstra was added for the per-node question and BFS was kept for hops and reachability.

Split Mesh

Prim assumes a graph is connected. Taking Moisture Sensor B and Pump B down together cuts off Greenhouse Temp, which only links to those two, so a naïve version would loop or throw. Prim was made to span only what the gateway can reach and return the rest as an unreachable list.

Joining C# backend to WPF frontend

Shared contract records in SmartX.Shared give both sides the same types, the 16 nodes are all drawn on a Canvas from a stored position instead of a layout engine, and the view is cached with its three-second poll running only while the tab is visible, so Part 3 does not slow the other pillars.

Proving the results

There is not a reference answer for a spanning tree, so the expected figures where checked with an independent calculation and they were fixed as expected values in the 42 xUnit test methods in SmartX.Tests, alongside the AVL balance, range queries and a 2000-node mesh.

Build failing on some machines

Downloading the repository as a ZIP and extracting it in Explorer nests the folder inside itself, and WPF's temporary XAML project then writes files whose paths pass Window's 260-character limit. Dotnet build copes with that but Visual Studio's MSBuild does note, so it looked like an IDE bug. A Directory.Build.props was added that moves the intermediate obj folder to a short path under LOCALAPPDATA only when the project folder is already deeper than 140 characters, so a normal clone builds exactly as before.

Debugging Approach

The clearest example of using a debugging approach was at restoring a failed node. When Pump A was switched off and back on, the result could report no links restored, because AlertTracker still listed the node as disconnected until its next packet arrived, so the algorithm kept treating it as if it was down. A test now covers it where it fails Pump A, marks it silent the way the health monitor would, restores it, and expects 4 links restored and 5 nodes re-routed. The fix was to let SetDown count the restored node as up when it works out the picture after the change. A failure was reduced to the smallest input that shows it, such as a three-node tree, checked by hand, and then returned to the full mesh.

Key Learnings

Learning	Where it came from
Choose structure by the access pattern, not by its name	Timestamps arrive in order, which would turn a plain binary search tree into a list, so a self-balancing tree was needed
An existing library type can hide a cost	The sorted collection was a balanced tree, but the way it was queried made range searches linear.
Count the work an algorithm does, not just the time	Returning the visited-node count made the tree's benefit measurable in the demo.
Different algorithms answer different questions	BFS gave fewest hops, Prim gave lowest energy, and the two routes for one sensor differed.
Design for the failure case first	The cut-off mesh forced Prim to handle any unreachable nodes
Keep a stable interface when changing internals	Parts 1 and 2 kept working because the stores public methods did not change
Verify with an independent calculation	Expected results were produced separately from the code that was tested

4. Technology Recommendations and Future Scaling

Smart-X currently runs on one API and one client against simulated sensors, with everything being stored in memory. These nine additions would turn it into a production IoT platform within the .NET ecosystem. Starting with a five-step continuous integration workflow (GitHub, n.d.)

Technology	Role in SmartX	Why it fits .NET	Benefit
GitHub Actions	Build and test every push and pull request	Setup-dotnet runs dotnet restore, build and test on hosted runners	Build that does not compile is caught before its submitted
Docker Compose	Extend the existing file, which already runs the API in one command, with an MQTT broker and database service.	Microsoft documents Compose for multi-container .NET apps with override files per environment	No specific setup, secrets kept in environment files
JWT bearer authentication	Protects the /api routes; today the API accepts any caller and any browser origin	Minimal APIs support it through the JwtBearer package and RequireAuthorization	Only signed-in operators can fail nodes or send commands
ASP.NET Core SignalR	Push topology and alert changes to the client instead of polling every 3 seconds	Part of ASP.NET Core, with a .NET client library	Updates arrive when they happen, and idle polling stops
MQTT telemetry ingestion	Let the device publish over MQTT, which ESP32 firmware already supports through IoT hub, with a background service calling Append	A hosted background service, like the existing queue consumer, subscribes and feeds the same ingestion pipeline	Real devices connect without HTTP gateway, using TLS and client certificates.
OpenTelemetry for .NET	Record range-query time, nodes visited and Prim run time as metrics and traces	Supports the ASP.NET Core minimal APIs	The tree and graph can be tuned from measured data.
Azure IoT Hub	Replace the simulator with real device ingestion	Device SDKs, and routing of telemetry to other Azure Services	Per-device access control and managed capacity
TimescaleDB	Keep history beyond the 5000 in-memory readings	A PostgreSQL extension, reachable from .NET through NpgSQL	Time partitioned storage keeps old data cheap to query and drop

Azure Contrainers Apps	Host the API containers with automatic scaling	Runs the same container images built for Compose	Scales on load and can scale to zero

Cloud Ingestion Pipeline

Sensors would publish to Azure IoT Hub, which routes telemetry to other Azure services and gives device twins, certificate-based security and Microsoft Entra ID access control (Microsoft, n.d.a). A small ingestion service would call the same Append metho that the simulator uses, so the tree, graph and page need no change. Devices can connect to IoT Hub over MQTT, but it only supports a limited part of the protocol (Microsoft, n.d.e), so a design that needs full broker features would add a separate broker. ESP32 firmware already includes an MQTT client that can authenticate with a client certificate over TLS (Espressif Systems, n.d.). The trade-off is running cost and reliance on cloud, against patching and securing that a self-hosted broker container needs

Security

Currently the API has no authentication, and its CORS policy allows any origin. This is fine on one laptop but not on a shared network, because anyone who can reach port 5224 can switch nodes off or send commands. ASP.NET Core minimal APIs support JWT bearer authentication through the Microsoft.AspNetCore.Authentication.JwtBearer Package (Microsoft, n.d.f), so requiring authorization on the route groups and having the WPF client send a token with each request would close that gap, while per-device certificates cover the device side. The trade-off is a token issuer to run or rent, such as Microsoft Entra ID, and a sign-in step.

Durable History

Memory is capped at 5000 reading per device. TimescaleDB divides a table into time-based chunks to improve insert and query performance (Timescale, n.d.), so it could hold the full history while the tree acts as a hot cache for recent windows.

Container Orchestration

Smart-X already ships a docker-compose.yml that start the API with one command and Compose can add a database and broker the same way (Microsoft, n.d.b). The same images

can then run on Azure Container Apps, which scales on HTTP traffic, event or KEDA triggers (Microsoft, n.d.c), once the topology services state moves to a shared store. Container Apps can scale to zero replicas, which saves cost when idle but adds cold start delay when work arrives (Microsoft, n.d.g), so the gateway should keep a minimum of one replica.

Telemetry Analysis and Push

SignalR pushes updates to clients instantly instead of polling every three seconds (Microsoft, n.d.d) and OpenTelemetry gives vendor-neutral traces and metrics (OpenTelemetry, n.d.) for timing each range query and spanning-tree run. In the client, TopologyView refreshes from a DispatcherTimer every three seconds, and a SignalR hub would replace that timer with a push each time MeshTopologyService changes state. SignalR connections are persistent, so running more than one API instance needs sticky sessions or Azure SignalR Service (Microsoft, n.d.h).

References

.NET Foundation, n.d.. *PriorityQueue.cs, dotnet/runtime source*. [Online]
Available at:
https://source.dot.net/System.Collections/System/Collections/Generic/PriorityQueue.cs.html
[Accessed 12 November 2026].

Adelson-Velsky, G. & Landis, E., 1962. An algorithm for the organization of information. *Doklady Akademii Nauk SSSR,* 146(2), p. 263–266.

Cormen, T., Leiserson, C., Rivest, R. & Stein, C., 2022. *Introduction to algorithms*. 4th ed. Cambridge, MA: MIT Press.

Dijkstra, E., 1959. A note on two problems in connexion with graphs. *Numerische Mathematik,* 1(1), p. 269–271.

Espressif Systems, n.d.. *ESP-MQTT*. [Online]
Available at: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/api-reference/protocols/mqtt.html
[Accessed 12 November 2026].

GitHub, n.d.. *Building and testing .NET*. [Online]
Available at: https://docs.github.com/en/actions/tutorials/build-and-test-code/net
[Accessed 12 November 2026].

Indeed Editorial Team, 2025. *Project Selection: Definition, Importance and Steps*. [Online]
Available at: https://www.indeed.com/career-advice/career-development/project-selection
[Accessed 26 August 2025].

Kruskal, J., 1956. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society,* 7(1), p. 48–50.

Microsoft, n.d.a. *Azure IoT Hub documentation*. [Online]
Available at: https://learn.microsoft.com/azure/iot-hub/iot-hub-devguide
[Accessed 12 November 2026].

Microsoft, n.d.b. *Defining your multi-container application with docker-compose.yml*. [Online]
Available at: https://learn.microsoft.com/en-us/dotnet/architecture/microservices/multi-container-microservice-net-applications/multi-container-applications-docker-compose
[Accessed 12 November 2026].

Microsoft, n.d.c. *Azure Container Apps overview*. [Online]
Available at: https://learn.microsoft.com/azure/container-apps/overview
[Accessed 12 November 2026].

Microsoft, n.d.d. *Overview of ASP.NET Core SignalR*. [Online]
Available at: https://learn.microsoft.com/aspnet/core/signalr
[Accessed 12 November 2026].

Microsoft, n.d.e. *Azure IoT Hub communication protocols and ports*. [Online]
Available at: https://learn.microsoft.com/en-us/azure/iot-hub/iot-hub-devguide-protocols
[Accessed 12 November 2026].

Microsoft, n.d.f. *Authentication and authorization in minimal APIs*. [Online]
Available at: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis/security
[Accessed 12 November 2026].

Microsoft, n.d.g. *Configure autoscaling for Durable Task SDK apps in Azure Container Apps*. [Online]
Available at: https://learn.microsoft.com/en-us/azure/durable-task/sdks/durable-task-scheduler-auto-scaling
[Accessed 12 November 2026].

Microsoft, n.d.h. *ASP.NET Core SignalR hosting and scaling*. [Online]
Available at: https://learn.microsoft.com/en-us/aspnet/core/signalr/scale
[Accessed 12 November 2026].

OctopusDeploy, 2026. *Making the most of your software environments*. [Online]
Available at: https://octopus.com/devops/software-deployments/software-environment/
[Accessed 12 August 2026].

OpenTelemetry, n.d.. *Getting started: .NET*. [Online]
Available at: https://opentelemetry.io/docs/languages/dotnet/getting-started/
[Accessed 12 November 2026].

Prim, R., 1957. Shortest connection networks and some generalizations. *Bell System Technical Journal,* 36(6), p. 1389–1401.

Sedgewick, R. & Wayne, K., 2011. *Algorithms*. 4th ed. Upper Saddle River, NJ: Addison-Wesley.

Timescale, n.d.. *About hypertables*. [Online]
Available at: https://docs.timescale.com/use-timescale/latest/hypertables/about-

hypertables
[Accessed 12 November 2026].